FastAPI Reference

Exceptions - `HTTPException` and `WebSocketException`

These are the exceptions that you can raise to show errors to the client.

When you raise an exception, as would happen with normal Python, the rest of the execution is aborted. This way you can raise these exceptions from anywhere in the code to abort a request and show the error to the client.

You can use:

- `HTTPException`
- `WebSocketException`

These exceptions can be imported directly from `fastapi`:

```
from fastapi import HTTPException, WebSocketException
```

`class` fastapi.HTTPException

```
HTTPException(status_code, detail=None, headers=None)
```

Bases: `HTTPException`

An HTTP exception you can raise in your own code to show errors to the client.

This is for client errors, invalid authentication, invalid data, etc. Not for server errors in your code.

Read more about it in the FastAPI docs for Handling Errors.

Example

```
from fastapi import FastAPI, HTTPException

app = FastAPI()

items = {"foo": "The Foo Wrestlers"}


@app.get("/items/{item_id}")
async def read_item(item_id: str):
    if item_id not in items:
        raise HTTPException(status_code=404, detail="Item not found")
    return {"item": items[item_id]}
```

PARAMETER	DESCRIPTION
`status_code`	HTTP status code to send to the client. Read more about it in the FastAPI docs for Handling Errors **TYPE:** `int`

Ask AI

PARAMETER	DESCRIPTION
`detail`	Any data to be sent to the client in the `detail` key of the JSON response. Read more about it in the FastAPI docs for Handling Errors **TYPE:** `Any` **DEFAULT:** `None`
`headers`	Any headers to send to the client in the response. Read more about it in the FastAPI docs for Handling Errors **TYPE:** `Mapping[str, str] | None` **DEFAULT:** `None`

Source code in `fastapi/exceptions.py`

```
def __init__(
    self,
    status_code: Annotated[
        int,
        Doc(
            """
            HTTP status code to send to the client.

            Read more about it in the
            [FastAPI docs for Handling Errors](https://fastapi.tiangolo.com/tutorial/handling-errors/#use-httpexception)
            """
        ),
    ],
    detail: Annotated[
        Any,
        Doc(
            """
            Any data to be sent to the client in the `detail` key of the JSON
            response.

            Read more about it in the
            [FastAPI docs for Handling Errors](https://fastapi.tiangolo.com/tutorial/handling-errors/#use-httpexception)
            """
        ),
    ] = None,
    headers: Annotated[
        Mapping[str, str] | None,
        Doc(
            """
            Any headers to send to the client in the response.

            Read more about it in the
            [FastAPI docs for Handling Errors](https://fastapi.tiangolo.com/tutorial/handling-errors/#add-custom-headers)

            """
        ),
    ] = None,
) -> None:
    super().__init__(status_code=status_code, detail=detail, headers=headers)
```

attr status_code `instance-attribute`

```
status_code = status_code
```

attr detail instance-attribute

```
detail = detail
```

attr headers instance-attribute

```
headers = headers
```

class fastapi.WebSocketException

```
WebSocketException(code, reason=None)
```

Bases: `WebSocketException`

A WebSocket exception you can raise in your own code to show errors to the client.

This is for client errors, invalid authentication, invalid data, etc. Not for server errors in your code.

Read more about it in the FastAPI docs for WebSockets.

Example

```
from typing import Annotated

from fastapi import (
    Cookie,
    FastAPI,
    WebSocket,
    WebSocketException,
    status,
)

app = FastAPI()

@app.websocket("/items/{item_id}/ws")
async def websocket_endpoint(
    *,
    websocket: WebSocket,
    session: Annotated[str | None, Cookie()] = None,
    item_id: str,
):
    if session is None:
        raise WebSocketException(code=status.WS_1008_POLICY_VIOLATION)
    await websocket.accept()
    while True:
        data = await websocket.receive_text()
        await websocket.send_text(f"Session cookie is: {session}")
        await websocket.send_text(f"Message text was: {data}, for item ID: {item_id}")
```

PARAMETER	DESCRIPTION
`code`	A closing code from the valid codes defined in the specification. **TYPE:** `int`
`reason`	The reason to close the WebSocket connection. It is UTF- 8 -encoded data. The interpretation of the reason is up to the application, it is not specified by the WebSocket specification.

PARAMETER	DESCRIPTION
	It could contain text that could be human-readable or interpretable by the client code, etc. **TYPE:** `str | None` **DEFAULT:** `None`

Source code in `fastapi/exceptions.py`

```
def __init__(
    self,
    code: Annotated[
        int,
        Doc(
            """
            A closing code from the
            [valid codes defined in the specification]
(https://datatracker.ietf.org/doc/html/rfc6455#section-7.4.1).
            """
        ),
    ],
    reason: Annotated[
        str | None,
        Doc(
            """
            The reason to close the WebSocket connection.

            It is UTF-8-encoded data. The interpretation of the reason is up to the
            application, it is not specified by the WebSocket specification.

            It could contain text that could be human-readable or interpretable
            by the client code, etc.
            """
        ),
    ] = None,
) -> None:
    super().__init__(code=code, reason=reason)
```

`attr` code `instance-attribute`

```
code = code
```

`attr` reason `instance-attribute`

```
reason = reason or ''
```